PRAKTIKUM SISTEM OPERASI

MODUL VI

Proses Sekuensial dan Konkuren

Oleh:

Christian Felix Saliman Sugiono (2311104031)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

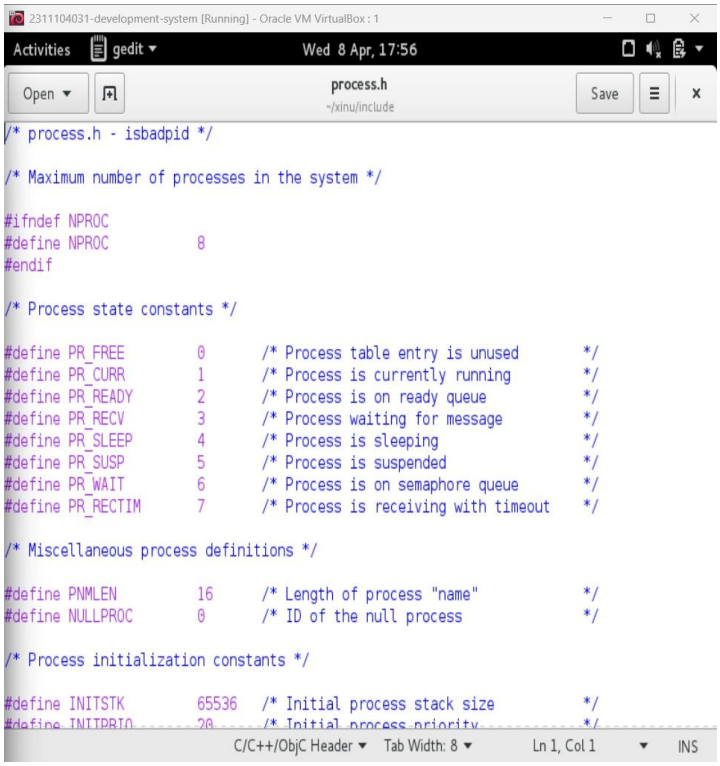
UNIVERSITAS TELKOM

2026

JURNAL

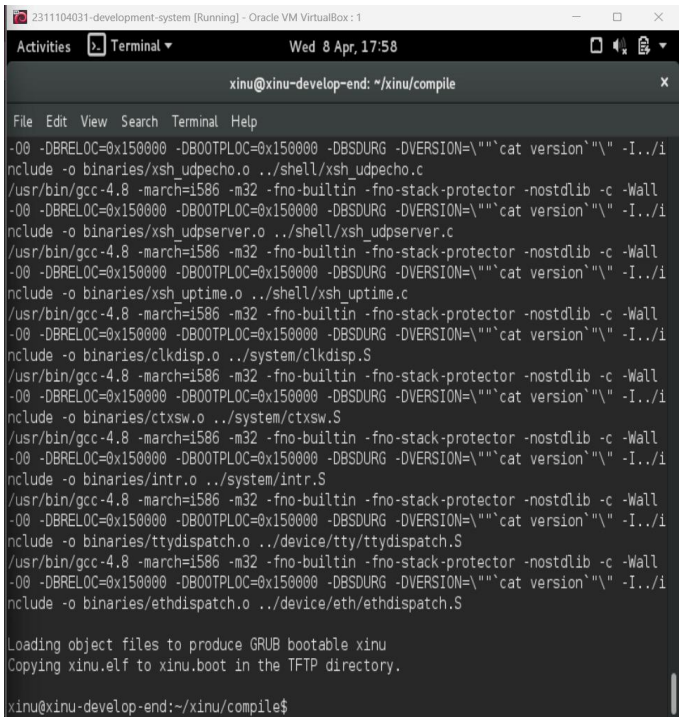
1. [10 Poin] Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah $cat /proc/sys/kernel/pid_max

   Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!



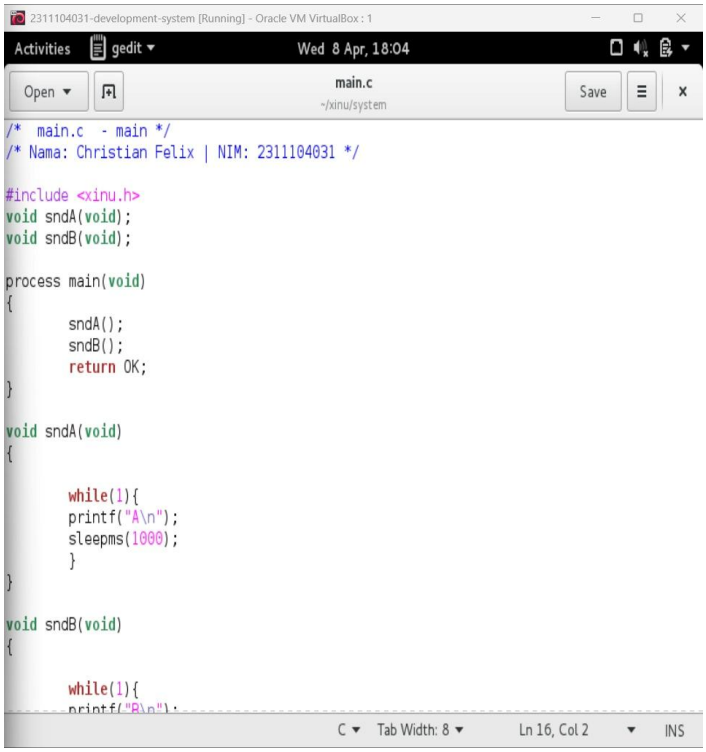
   Before edit

After edit 150 processes

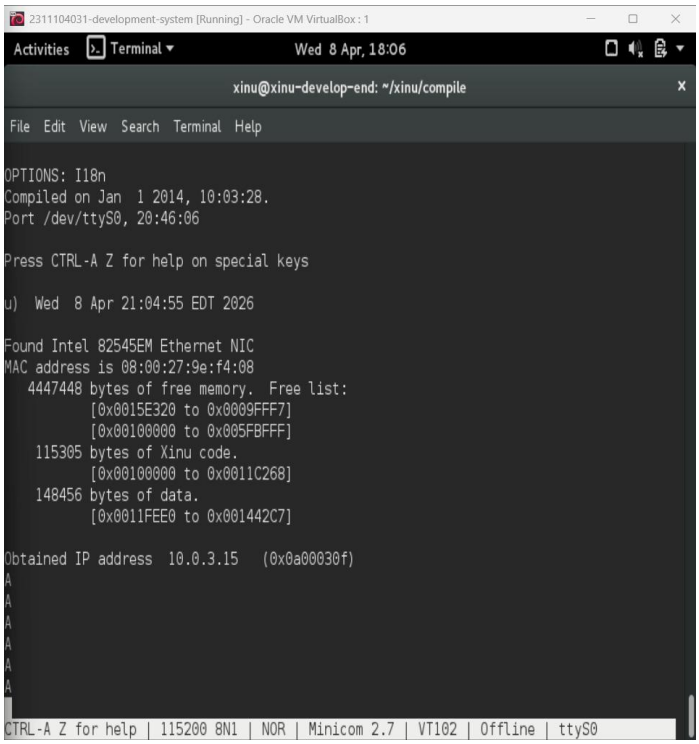


Hasil compile dari make

2. [20 Poin] Jalankan kode main_ex1!



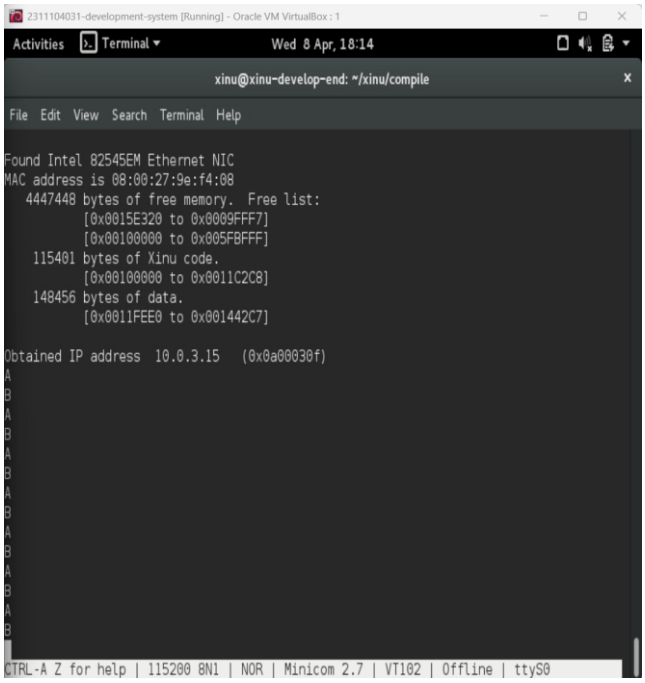
Edit file main.c dengan main_ex1



Hasil running main_ex1

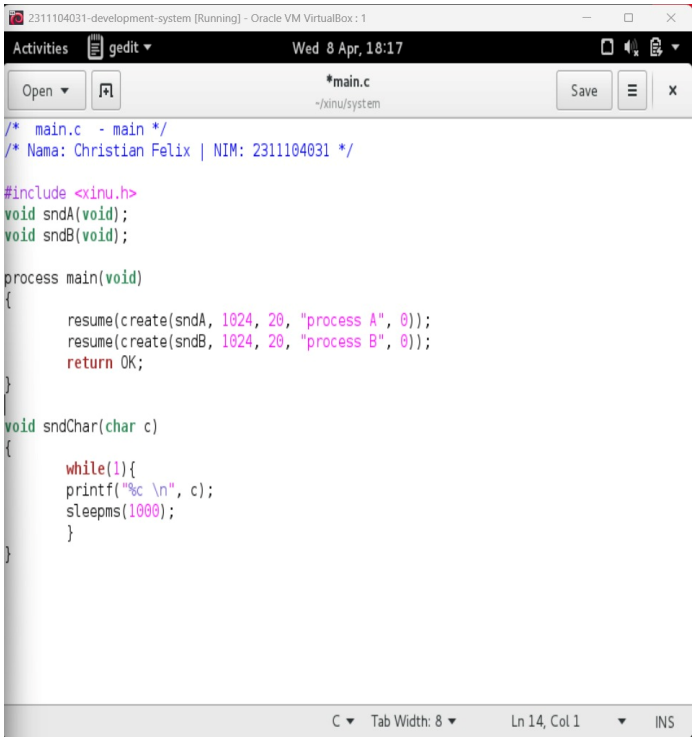
3. [20 Poin] Jalankan kode main_ex2!

Edit file main dengan main_ex2

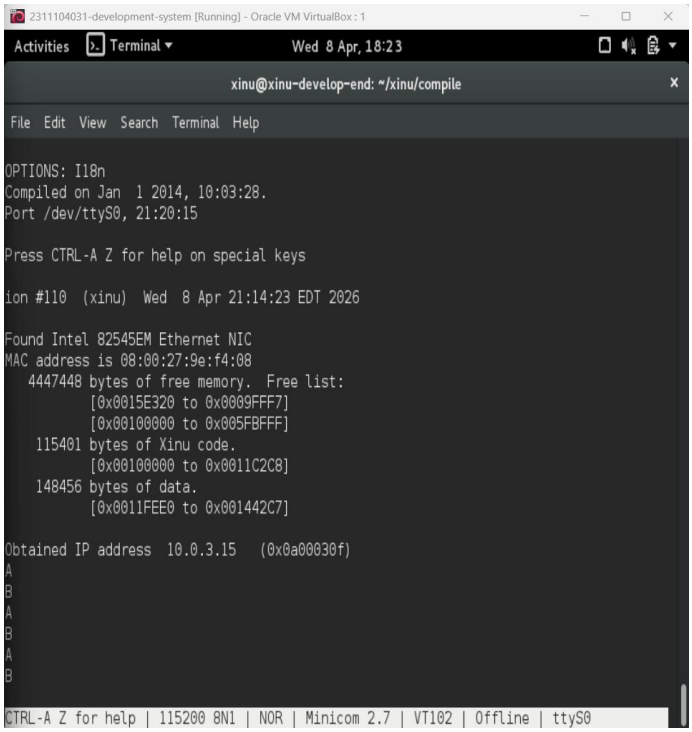


Hasil running dengan main_ex2

4. [20 Poin] Jalankan kode main_ex3!



Edit file dengan main_ex3



Hasil running dengan main_ex3

5. [30 Poin] Buatlah 2 proses produser dan konsumer. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

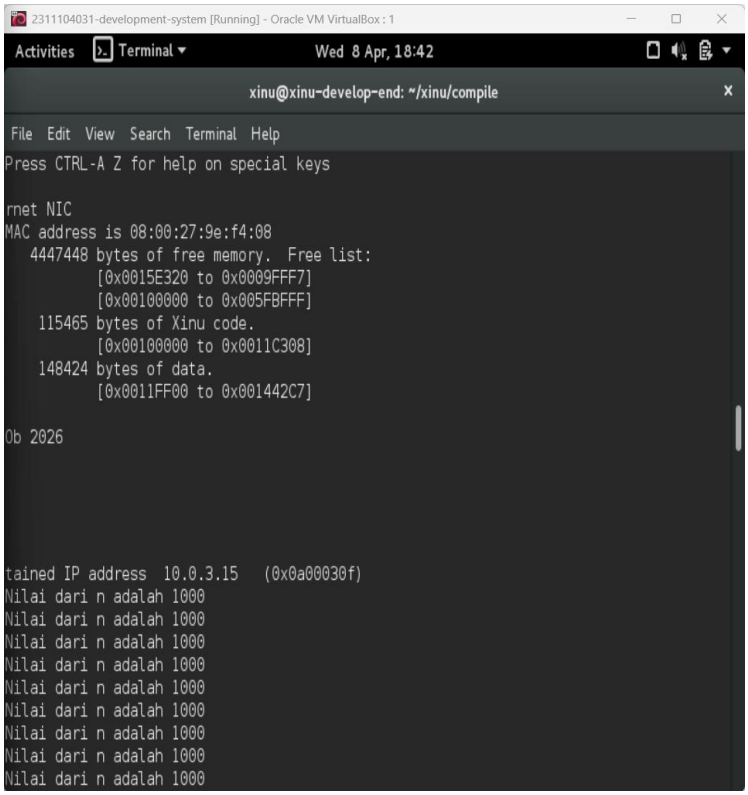
```
int32 n = 0; //global variabel

void produser(void){
      int32 i;
      for (i=1; i<=1000; i++){
            n++;
      }
}

void konsumer(void){
      int32 i;
      for (i=1; i<=1000; i++){
            printf("Nilai dari n adalah %d\n",n);
      }
}
```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal). Jelaskan mengapa hasilnya seperti itu!

Pengeditan file main.c dengan kode diatas



Hasil running main.c dengan kode baru

Penjelasan mengapa terjadi seperti diatas: Output menampilkan nilai n = 1000 sebanyak 1000 kali. Hal ini terjadi karena scheduler Xinu memberikan giliran CPU kepada proses produser terlebih dahulu hingga selesai (n mencapai 1000), baru kemudian proses konsumer mendapat giliran dan membaca nilai n yang sudah final yaitu 1000. Ini merupakan salah satu kemungkinan hasil dari race condition — hasilnya tidak deterministik dan bergantung pada keputusan scheduler. Pada kondisi lain, output bisa berupa nilai acak atau bahkan semua 0 jika konsumer mendapat giliran lebih dulu